

TÉCNICAS DE HACKING

Práctica 2

Reconocimiento Activo

Asignatura

Técnicas de Hacking

Profesor

Alfredo Robledano

Grado

Ingeniería Informática en Ciberseguridad

Entorno

Kali Linux (VM) + Docker

Alumno

Arturo Fernández Merino

Índice de contenidos

1. Introducción	3
2. Desarrollo	4
2.1. Descubrimiento de hosts con Scapy	4
2.1.1. La función <code>craft_discovery_pkts</code>	4
2.1.2. Entorno de pruebas	6
2.2. Comportamiento por defecto de nmap y estado de puertos	6
2.2.1. Estado de un puerto	6
2.2.2. Comportamiento por defecto de nmap	7
3. Resultados	8
3.1. Host discovery con Scapy	8
3.2. Escaneo de puertos con nmap	9
4. Conclusión	11
5. Bibliografía	12

1. Introducción

El **reconocimiento activo** es la fase de una auditoría en la que el atacante envía estímulos a la red objetivo y analiza las respuestas para inferir qué dispositivos están vivos y qué servicios exponen. A diferencia del reconocimiento pasivo, aquí sí se genera tráfico dirigido al objetivo, por lo que es una técnica más ruidosa pero también más precisa.

Esta práctica se divide en dos bloques. En el primero se implementa una herramienta propia de *host discovery* en Python con la librería Scapy, apoyada en tres estímulos independientes —UDP, TCP (ACK) e ICMP (timestamp)— y se utiliza para detectar hosts activos. En el segundo se analiza, mediante un *packet sniffer*, el comportamiento por defecto de nmap en el reconocimiento de puertos y su estado.

Todo el trabajo se realiza sobre un entorno virtualizado y contenerizado, conforme a las restricciones de la práctica: una máquina **Kali Linux** como atacante y un laboratorio de contenedores **Docker** que hace las veces de red objetivo.

2. Desarrollo

2.1. Descubrimiento de hosts con Scapy

2.1.1. La función `craft_discovery_pkts`

La función construye y devuelve una **lista de paquetes** lista para enviar con `sr()`, separando deliberadamente el *crafteo* del *envío*. De este modo la misma función puede reutilizarse en distintos scripts (envío síncrono con `sr`, asíncrono, por lotes, etc.) sin acoplarla a una forma concreta de transmisión.

Sus argumentos son:

- **protocolos (obligatorio)**: admite un `str` o una lista de `str`. Internamente se normaliza a una lista en mayúsculas y se valida contra el conjunto `{UDP, TCP, ICMP}`, lanzando `ValueError` ante cualquier protocolo no soportado.
- **objetivos (obligatorio)**: una IP única o un rango en formato Scapy (p.ej. `"172.28.0.0/24"`). Se pasa tal cual a `IP(dst=...)`; Scapy expande el rango de forma perezosa en el momento del envío.
- **num_pkts (opcional)**: diccionario `{protocolo: nº de paquetes}`. Si no se pasa, se craftea un único paquete de cada tipo exigido; si el diccionario existe pero le falta una clave concreta, se asume 1 para ese protocolo.
- **puerto (opcional)**: puerto de capa 4 (L4) para TCP y UDP; por defecto, el 80.

Cada estímulo se apoya en un comportamiento distinto de la pila para revelar un host vivo:

- **ICMP timestamp (type=13)**: un host que lo soporta responde con un *timestamp reply* (type=14).
- **TCP ACK (flags="A")**: al no existir una conexión previa establecida, el host responde con un `RST`.
- **UDP a un puerto cerrado**: el host responde con un `ICMP port-unreachable` (type 3, code 3).

En los tres casos, cualquier respuesta delata la IP del emisor en el campo `respuesta.src`, que es justo lo que recolecta la función auxiliar `hosts_activos`.

```
#!/usr/bin/env python3
"""
Host discovery con Scapy usando UDP, TCP (ACK) e ICMP (timestamp).
Requiere privilegios de root para enviar paquetes en crudo (sudo).
"""

from scapy.all import IP, TCP, UDP, ICMP, sr

PROTOS_VALIDOS = {"UDP", "TCP", "ICMP"}

def craft_discovery_pkts(protocolos, objetivos, num_pkts=None, puerto=80):
    """
    Construye paquetes de host discovery y los devuelve en una lista.

    protocolos : str o lista de str -> "TCP", "UDP", "ICMP" (hasta 3) [obligatorio]
    objetivos   : str o rango scapy -> "192.168.1.1", "10.0.0.0/24" [obligatorio]
    num_pkts    : dict opcional     -> {"TCP": 2, "UDP": 1}; por defecto 1 de cada
    """
```

```

puerto : int opcional -> puerto L4 para TCP/UDP (defecto 80)
"""

# Normalizar protocolos a lista de mayúsculas (acepta str o lista)
if isinstance(protocolos, str):
    protocolos = [protocolos]
protocolos = [p.upper() for p in protocolos]

for p in protocolos:
    if p not in PROTOS_VALIDOS:
        raise ValueError(f"Protocolo no válido: {p} (usa {PROTOS_VALIDOS})")

if num_pkts is None:
    num_pkts = {}

paquetes = []
for proto in protocolos:
    cantidad = num_pkts.get(proto, 1) # si falta la clave -> 1 paquete
    for _ in range(cantidad):
        if proto == "ICMP":
            # type=13 -> ICMP Timestamp Request (respuesta viva = type 14)
            pkt = IP(dst=objtivos) / ICMP(type=13)
        elif proto == "TCP":
            # flags="A" -> ACK; un host vivo responde con RST
            pkt = IP(dst=objtivos) / TCP(dport=puerto, flags="A")
        else: # UDP
            # puerto cerrado -> ICMP port-unreachable = host vivo
            pkt = IP(dst=objtivos) / UDP(dport=puerto)
        paquetes.append(pkt)
    return paquetes

def hosts_activos(respuestas):
    """Extrae el conjunto de IPs que han respondido (host vivo)."""
    return {recibido.src for _, recibido in respuestas}

if __name__ == "__main__":
    # --- AJUSTA estas IPs a tu red/laboratorio ---
    HOST_ACTIVO = "172.28.0.10" # p.ej. tu gateway o una VM que sabes viva
    HOST_INACTIVO = "172.28.0.99" # IP libre del subnet: sin host garantizado

    # 1) Envío con los 3 protocolos a un host que debería estar activo
    pkts_vivo = craft_discovery_pkts(
        ["ICMP", "TCP", "UDP"], HOST_ACTIVO,
        num_pkts={"ICMP": 1, "TCP": 1, "UDP": 1}, puerto=80,
    )

    # 2) Envío (solo ICMP, como str) a una IP sin host activo
    pkts_muerto = craft_discovery_pkts("ICMP", HOST_INACTIVO)

    todos = pkts_vivo + pkts_muerto
    print(f"[*] Enviando {len(todos)} paquetes...")

    ans, unans = sr(todos, timeout=2, verbose=0)

    activos = hosts_activos(ans)
    print(f"[+] Hosts activos detectados: {len(activos)}")

```

```

for ip in sorted(activos):
    print(f" {ip}")

if not activos:
    print("[-] Nadie respondió (¿firewall, permisos root o ICMP filtrado?)")

```

2.1.2. Entorno de pruebas

El laboratorio se define con Docker Compose sobre una red *bridge* aislada (`172.28.0.0/24`). Se despliegan dos hosts activos y se deja una IP libre a propósito para disponer del caso de «host inactivo» que exige el enunciado. El objetivo multi-servicio (`lab-target`) es una imagen mínima de Alpine que levanta `sshd` y `nginx` , exponiendo por tanto dos puertos abiertos (22 y 80) para la segunda parte.

Host	IP	Estado	Servicios
lab-target	172.28.0.10	activo	SSH (22), HTTP (80)
lab-web	172.28.0.20	activo	HTTP (80)
(libre)	172.28.0.99	inactivo	— (caso de host muerto)

Tabla 1: Topología del laboratorio de contenedores.

```

services:
  # Host multi-servicio (SSH + HTTP) -> objetivo del escaneo de puertos (Parte 2)
  target:
    build: { context: ., dockerfile: Dockerfile.target }
    container_name: lab-target
    cap_add: [NET_ADMIN]
    networks: { labnet: { ipv4_address: 172.28.0.10 } }

  # Host solo-web -> segundo host activo para el host discovery (Parte 1)
  web:
    image: nginx:alpine
    container_name: lab-web
    networks: { labnet: { ipv4_address: 172.28.0.20 } }

# La IP 172.28.0.99 queda LIBRE a propósito -> caso "host inactivo"
networks:
  labnet:
    driver: bridge
    ipam: { config: [{ subnet: 172.28.0.0/24 }] }

```

2.2. Comportamiento por defecto de nmap y estado de puertos

2.2.1. Estado de un puerto

El **estado de un puerto** es la conclusión a la que llega el escáner tras enviar un estímulo y observar (o no) la respuesta. Depende de si hay un servicio escuchando y de si existe filtrado por el camino. Para un *SYN scan* los estados relevantes son:

Estado	Significado	Estímulo → Respuesta
Abierto	Hay un servicio escuchando y aceptando conexiones.	SYN → SYN/ACK ; nmap responde RST y no completa el <i>handshake</i> .
Cerrado	El host está vivo pero ningún servicio escucha en ese puerto.	SYN → RST/ACK .
Filtrado	Un cortafuegos descarta el estímulo; nmap no puede decidir.	SYN → (sin respuesta) o ICMP unreachable (type 3).

Tabla 2: Estados de un puerto y patrón estímulo/respuesta en un SYN scan.

2.2.2. Comportamiento por defecto de nmap

Ejecutado como root, el escaneo por defecto de nmap es un **SYN scan** (`-sS`), también llamado *half-open* porque no completa el 3-way handshake. Sin privilegios, recurre a un **connect scan** (`-sT`), que sí abre la conexión completa mediante la llamada `connect()` . Por defecto nmap no barre los 65535 puertos, sino los **1000 puertos más comunes** según su base de datos `nmap-services` , con una temporización `-T3` y retransmisiones cuando un puerto no responde.

Nmap, por defecto, realiza además descubrimiento de hosts (*ping*) y resolución DNS. Como esta parte se centra únicamente en el reconocimiento de puertos, ese tráfico se descarta lanzando el escaneo con `-Pn` (omitir descubrimiento) y `-n` (omitir DNS).

3. Resultados

3.1. Host discovery con Scapy

Tras levantar el laboratorio, se lanza `host_discovery.py` contra un host activo (172.28.0.10) y contra la IP libre (172.28.0.99), capturando el tráfico con `tcpdump` en la interfaz del *bridge*. La salida del script identifica correctamente los hosts vivos y descarta la IP sin host.

```
(xyzz0@kali)-[~/Desktop/HT/P2]
$ sudo python3 host-discovery.py

[sudo] password for xyzz0:
[*] Enviando 4 paquetes...
[+] Hosts activos detectados: 1
    172.28.0.10
```

Figura 1: Salida del script: se detectan como activos los hosts que responden, mientras que la IP 172.28.0.99 no genera respuesta alguna.

La captura completa en Wireshark muestra el conjunto de estímulos enviados y las respuestas recibidas, que analizamos a continuación estímulo por estímulo.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	02:42:bf:b4:a2:e2	Broadcast	ARP	42	who has 172.28.0.10? Tell 172.28.0.1
2	0.000044	02:42:ac:1c:00:0a	02:42:bf:b4:a2:e2	ARP	42	172.28.0.10 is at 02:42:ac:1c:00:0a
3	0.011679	172.28.0.1	172.28.0.10	ICMP	54	Timestamp request id=0x0000, seq=0/0, ttl=64
4	0.011745	172.28.0.10	172.28.0.1	ICMP	54	Timestamp reply id=0x0000, seq=0/0, ttl=64
5	0.012313	172.28.0.1	172.28.0.10	TCP	54	[TCP Window Update] 20 → 80 [ACK] Seq=1 Ack=1 Win=8192 Len=0
6	0.012333	172.28.0.10	172.28.0.1	TCP	54	80 → 20 [RST] Seq=1 Win=0 Len=0
7	0.013171	172.28.0.1	172.28.0.10	UDP	42	53 → 80 Len=0
8	0.013193	172.28.0.10	172.28.0.1	ICMP	70	Destination unreachable (Port unreachable)
9	5.231318	02:42:ac:1c:00:0a	02:42:bf:b4:a2:e2	ARP	42	who has 172.28.0.1? Tell 172.28.0.10
10	5.231332	02:42:bf:b4:a2:e2	02:42:ac:1c:00:0a	ARP	42	172.28.0.1 is at 02:42:bf:b4:a2:e2

Figura 2: Vista general de la captura del *host discovery* en Wireshark.

Firma 1 — ICMP timestamp

Estímulo ICMP type=13 (*timestamp request*). Un host vivo responde con ICMP type=14 (*timestamp reply*); la presencia del reply confirma el host activo.

icmp.type==13 or icmp.type==14						
No.	Time	Source	Destination	Protocol	Length	Info
3	0.011679	172.28.0.1	172.28.0.10	ICMP	54	Timestamp request id=0x0000, seq=0/0, ttl=64
4	0.011745	172.28.0.10	172.28.0.1	ICMP	54	Timestamp reply id=0x0000, seq=0/0, ttl=64

Figura 3: Filtro `icmp.type==13 or icmp.type==14` : petición y respuesta de timestamp.

Firma 2 — TCP ACK

Estímulo TCP con flags="A" (ACK) al puerto 80. Al no existir conexión previa, el host vivo responde con un RST, lo que delata su presencia.

tcp.flags.ack==1 and tcp.flags.syn==0						
No.	Time	Source	Destination	Protocol	Length	Info
5	0.012313	172.28.0.1	172.28.0.10	TCP	54	[TCP Window Update] 20 → 80 [ACK] Seq=1 Ack=1 Win=8192 Len=0

Figura 4: Filtro `tcp.flags.ack==1 and tcp.flags.syn==0` : ACK enviado y RST de respuesta.

Firma 3 — UDP

Estímulo UDP a un puerto cerrado. El host vivo responde con un ICMP port-unreachable (type 3, code 3), indicando que la máquina está activa aunque el puerto no ofrezca servicio.

udp.port==80 or icmp.type==3						
No.	Time	Source	Destination	Protocol	Length	Info
7	0.013171	172.28.0.1	172.28.0.10	UDP	42	53 → 80 Len=0
8	0.013193	172.28.0.10	172.28.0.1	ICMP	70	Destination unreachable (Port unreachable)

Figura 5: Filtro `udp.port==80 or icmp.type==3` : datagrama UDP y su *port-unreachable*.

3.2. Escaneo de puertos con nmap

Se lanza el escaneo por defecto descartando descubrimiento y DNS (`nmap -Pn -n 172.28.0.10`) para observar únicamente el reconocimiento de puertos. Nmap identifica los dos servicios expuestos por el objetivo.

```

└─$ sudo nmap -Pn -n 172.28.0.10
[sudo] password for xyzz0:
Starting Nmap 7.98 ( https://nmap.org ) at 2026-07-05 17:46 -0400
Nmap scan report for 172.28.0.10
Host is up (0.0000030s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
MAC Address: 02:42:AC:1C:00:0A (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 0.18 seconds

```

Figura 6: Salida de nmap: puertos 22 (SSH) y 80 (HTTP) abiertos en el objetivo.

En la captura se comprueba el **conteo de puertos**: nmap envía un segmento SYN a cada uno de los 1000 puertos más comunes —comportamiento por defecto—, en lugar de barrer el espacio completo de 65535 puertos. En cuanto al **conteo de paquetes**, se aprecian retransmisiones de SYN hacia los puertos que no responden.

Firma 4 — Puerto abierto

Patrón SYN → SYN/ACK → RST . Ante un puerto con servicio (22 y 80), el objetivo responde SYN/ACK ; nmap contesta con RST para no completar el handshake (característico del *half-open SYN scan*).

tcp.flags.syn==1 and tcp.flags.ack==0						
No.	Time	Source	Destination	Protocol	Length	Info
23	0.074424	172.28.0.1	172.28.0.10	TCP	58	35406 → 5900 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
30	0.076430	172.28.0.1	172.28.0.10	TCP	58	35406 → 199 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
32	0.076437	172.28.0.1	172.28.0.10	TCP	58	35406 → 111 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
34	0.076443	172.28.0.1	172.28.0.10	TCP	58	35406 → 53 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
36	0.076449	172.28.0.1	172.28.0.10	TCP	58	35406 → 507 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
38	0.076455	172.28.0.1	172.28.0.10	TCP	58	35406 → 8880 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
40	0.076461	172.28.0.1	172.28.0.10	TCP	58	35406 → 8880 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
42	0.076467	172.28.0.1	172.28.0.10	TCP	58	35406 → 143 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
44	0.076473	172.28.0.1	172.28.0.10	TCP	58	35406 → 21 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
46	0.076478	172.28.0.1	172.28.0.10	TCP	58	35406 → 135 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
48	0.076485	172.28.0.1	172.28.0.10	TCP	58	35406 → 3389 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
50	0.076491	172.28.0.1	172.28.0.10	TCP	58	35406 → 995 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
52	0.076497	172.28.0.1	172.28.0.10	TCP	58	35406 → 1025 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
54	0.076503	172.28.0.1	172.28.0.10	TCP	58	35406 → 1720 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
56	0.076509	172.28.0.1	172.28.0.10	TCP	58	35406 → 445 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
58	0.076515	172.28.0.1	172.28.0.10	TCP	58	35406 → 69 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
60	0.076521	172.28.0.1	172.28.0.10	TCP	58	35406 → 1110 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
62	0.076527	172.28.0.1	172.28.0.10	TCP	58	35406 → 7490 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
64	0.076533	172.28.0.1	172.28.0.10	TCP	58	35406 → 1839 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
66	0.076539	172.28.0.1	172.28.0.10	TCP	58	35406 → 3901 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
68	0.076545	172.28.0.1	172.28.0.10	TCP	58	35406 → 1863 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
70	0.076551	172.28.0.1	172.28.0.10	TCP	58	35406 → 6112 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
72	0.076557	172.28.0.1	172.28.0.10	TCP	58	35406 → 1024 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
74	0.076563	172.28.0.1	172.28.0.10	TCP	58	35406 → 49999 [SYN] Seq=0 Win=1024 Len=0 MSS=1460

Figura 7: Reconocimiento del puerto 80: estímulo SYN y respuesta SYN/ACK (puerto abierto).

tcp.flags.syn==1 and tcp.flags.ack==1						
No.	Time	Source	Destination	Protocol	Length	Info
16	0.076353	172.28.0.10	172.28.0.1	TCP	58	22 → 35406 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460
59	0.076520	172.28.0.10	172.28.0.1	TCP	58	80 → 35406 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460

Figura 8: Detalle del intercambio sobre el puerto 80 durante el escaneo.

Firma 5 — Puerto cerrado

Patrón `SYN → RST/ACK`. Ante un puerto sin servicio, el host —que está vivo— responde directamente con `RST`, permitiendo a `nmap` clasificarlo como *closed*.

tcp.flags.reset==1						
No.	Time	Source	Destination	Protocol	Length	Info
4	0.076294	172.28.0.10	172.28.0.1	TCP	54	1723 → 35406 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
6	0.076306	172.28.0.10	172.28.0.1	TCP	54	25 → 35406 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
8	0.076313	172.28.0.10	172.28.0.1	TCP	54	139 → 35406 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
10	0.076322	172.28.0.10	172.28.0.1	TCP	54	113 → 35406 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
12	0.076329	172.28.0.10	172.28.0.1	TCP	54	443 → 35406 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
14	0.076335	172.28.0.10	172.28.0.1	TCP	54	554 → 35406 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
17	0.076356	172.28.0.1	172.28.0.10	TCP	54	35406 → 22 [RST] Seq=1 Win=0 Len=0
19	0.076364	172.28.0.10	172.28.0.1	TCP	54	23 → 35406 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
21	0.076373	172.28.0.10	172.28.0.1	TCP	54	993 → 35406 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
23	0.076380	172.28.0.10	172.28.0.1	TCP	54	110 → 35406 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
25	0.076415	172.28.0.10	172.28.0.1	TCP	54	256 → 35406 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
27	0.076421	172.28.0.10	172.28.0.1	TCP	54	3306 → 35406 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
29	0.076427	172.28.0.10	172.28.0.1	TCP	54	5900 → 35406 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
31	0.076433	172.28.0.10	172.28.0.1	TCP	54	199 → 35406 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
33	0.076440	172.28.0.10	172.28.0.1	TCP	54	111 → 35406 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
35	0.076446	172.28.0.10	172.28.0.1	TCP	54	53 → 35406 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
37	0.076451	172.28.0.10	172.28.0.1	TCP	54	587 → 35406 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
39	0.076458	172.28.0.10	172.28.0.1	TCP	54	8080 → 35406 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
41	0.076464	172.28.0.10	172.28.0.1	TCP	54	8888 → 35406 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
43	0.076470	172.28.0.10	172.28.0.1	TCP	54	143 → 35406 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
45	0.076475	172.28.0.10	172.28.0.1	TCP	54	21 → 35406 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
47	0.076481	172.28.0.10	172.28.0.1	TCP	54	135 → 35406 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
49	0.076488	172.28.0.10	172.28.0.1	TCP	54	3389 → 35406 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0

Figura 9: Respuesta `RST` del objetivo ante el sondeo de un puerto cerrado.

4. Conclusión

Se ha implementado una herramienta de *host discovery* funcional y modular que, mediante tres estímulos independientes (ICMP timestamp, TCP ACK y UDP), distingue correctamente los hosts activos de las IPs sin host. Las capturas evidencian, en cada caso, el par estímulo/respuesta esperado: *timestamp reply* para ICMP, *RST* para el ACK y *port-unreachable* para UDP.

En la segunda parte se ha caracterizado el comportamiento por defecto de nmap: un SYN scan *half-open* sobre los 1000 puertos más comunes, con el patrón *SYN → SYN/ACK → RST* para los puertos abiertos y *SYN → RST* para los cerrados. El uso de *-Pn -n* ha permitido aislar el tráfico de reconocimiento de puertos del de descubrimiento de hosts, tal y como pedía el enunciado. El conjunto valida tanto la implementación propia como la comprensión del comportamiento de una herramienta estándar de la industria.

La gran diferencia es la calidad de los escaneos, en uno más personalizado generamos una cantidad de ruido bastante menor la cual es adaptada al contexto y al entorno, sin embargo mediante el uso de herramientas como NMAP, el uso general genera una cantidad excesiva de tráfico, ruido y escaneos innecesarios los cuales pueden llegar a ser desconocidos para alguien inexperto y causar complicaciones o problemas.

5. Bibliografía

- Nmap Project. **Nmap Reference Guide (Man Page)**. nmap.org/book/man.html
- Lyon, G. **Nmap Network Scanning — Port Scanning Techniques**. nmap.org/book/scan-methods.html
- Biondi, P. y colaboradores. **Scapy Documentation**. scapy.readthedocs.io
- Postel, J. (1981). **RFC 792 — Internet Control Message Protocol**. rfc-editor.org/rfc/rfc792
- Eddy, W. (2022). **RFC 9293 — Transmission Control Protocol**. rfc-editor.org/rfc/rfc9293